

Research Computing Documentation

Overview ▼

Tutorials ▲

ColdFront Tutorial ([coldfront_tutorial.html](#))

Linux & Bash Tutorial ([bash_tutorial.html](#))

SSH Tutorial ([ssh_tutorial.html](#))

Slurm Tutorial 1: Getting Started ([slurm_quick_start_tutorial.html](#))

Slurm Tutorial 2: Scaling Up ([slurm_tutorial_2.html](#))

Software Tutorial ([software_tutorial.html](#))

Storage Tutorial ([storage_tutorial.html](#))

Globus Tutorial ([globus_tutorial.html](#))

Quick Reference ▼

Misc Instructions ▼

Resources & Links ▼

Software Tutorial

Table of Contents

- [1 - Introduction](#)
 - [1.1 - Environments](#)
 - [1.2 - Software Support](#)
- [2 - Spack](#)
 - [2.1 - Spack Environments](#)
 - [2.2 - Spack via Interactive](#)
 - [2.3 - Spack via Batch](#)
 - [2.4 - Further Reading on Spack](#)
- [3 - Apptainer](#)
 - [3.1 - Containers](#)
 - [3.2 - Inside a Container Shell](#)
 - [3.3 - Apptainer via Interactive](#)
 - [3.4 - Apptainer via Batch](#)
 - [3.4 - Further Reading on Apptainer](#)

- [4 - Home Directory Installs](#)
 - [4.1 - Conda](#)
 - [4.2 - Conda via Interactive](#)
 - [4.3 - Conda via Batch](#)
 - [4.4 - Further Reading on Conda](#)
- [5 - Licensed Software](#)
- [6 - Summary](#)
 - [6.1 - Should I Use Spack, Apptainer, or Home Directory Installs?](#)
- [7 - Special Notes](#)
 - [7.1 - Mixing Spack and Conda](#)
 - [7.2 - Spanning Multiple Nodes](#)

In this tutorial, you will be introduced to the different ways you can access the software you need on the Research Computing cluster. We will discuss pros and cons for each approach.

This tutorial assumes you are familiar with Slurm and have written an `sbatch` script before. You can learn how to use Slurm by following along with **[Part 1 of our Slurm Tutorial \(slurm_quick_start_tutorial.html\)](#)**.

1 - Introduction

When you log into the cluster, only the software needed to run the cluster is loaded. None of the software you need for your research is loaded, so this tutorial will walk you through three different ways to load your research software: Spack, Apptainer, and Home Directory Installs. We will discuss these in detail throughout this tutorial, but first, let's talk about *environments*.

1.1 - Environments

So, what is an environment?

Think of your computer like a tool shed. It has all of the tools (software) you need. On the cluster, you have to share the tools in the tool shed, but your tools might not work well with other researchers' tools. So what you need is your own toolbox (environment) that you can place only the tools you need inside. You have access to your toolbox, and no one else does. If you need both an artist's toolbox and a welder's toolbox, you can create a toolbox for each. When you're done working on art, you can switch your toolbox and start welding.

It's not a perfect metaphor, but you get the idea. The problem with software is that you may need Python3.7, but another researcher needs Python3.6. Normally, you can't have both Python3.6 and Python3.7 installed at the same time... unless you install them in their own environments.

Spack, Apptainer, and Home Directory Installs allow you to access an environment that has the software you need for your research. We will come back to environments later.

1.2 - Software Support

Before we dive into Spack, Apptainer, and Home Directory Installs, we need to discuss what level of support Research Computing provides for each.

Spack is *fully supported* by Research Computing, which means we will package, build, install, test, and maintain software libraries and environments using Spack. Spack is a standard for HPC systems and prevents many of the challenges typically encountered with Home Directory Installs. We highly encourage you to utilize spack environments, as they are fully maintained and tested by the Research Computing team, allowing you to focus on your research instead of debugging build errors. However, there is one **very important** thing to note about spack: **Building and installing spack environments can take some time**, especially if you need us to package/test libraries that aren't already available in Spack. We can't guarantee your environment will be available on a specific timeline.

Apptainer is *minimally supported* by Research Computing, which means we will ensure that our installation of Apptainer is working properly and address any issues that arise, but we will not help you design or build containers, install software inside of containers, or troubleshoot software inside of containers. If we think of containers like a forest, Research Computing provides the land (Apptainer), but **you are responsible** for acquiring seeds (designing containers), planting seeds (building containers), watering the plants (installing software inside containers), and pruning branches (troubleshooting inside of containers). **Note:** When we say *Apptainer*, we are referring to a specific piece of software that runs containers; when we say *container*, we are referring to bundles of software that run inside of Apptainer.

Home Directory Installs are *not supported* by Research Computing, which means we provide your home directory, but we will not assist you with designing, building, installing, troubleshooting, and maintaining Home Directory Installs. **You are supporting any software you install yourself.**

Here is a summary of our support for these three approaches:

	Spack	Apptainer	Home Directory Installs
Support Level	Fully Supported by RC	Minimally Supported by RC	No Support from RC
Who Builds Software?	RC Team	You	You
Who Installs Software?	RC Team	You	You
Who Troubleshoots Software?	RC Team	You	You
Who Builds Environments?	RC Team	You	You

	Spack	Apptainer	Home Directory Installs
Who Maintains Environments?	RC Team	You	You

Spack is our only supported package manager on the cluster. Building, installing, and testing Spack environments can take some time; we cannot guarantee your environment will be available on a specific timeline.

Apptainer is installed and maintained by the RC team. **You are responsible for designing, building, installing, troubleshooting, and maintaining containers.**

Home Directory Installs are not supported by the RC team. **You are responsible for designing, building, installing, troubleshooting, and maintaining software in your home directory.**

2 - Spack

[Spack](#)  is our supported package manager on the cluster because it:

- Is a standard for HPC clusters
- Handles dependency conflicts
- Allows multiple versions of the same software to be installed
- Allows us to build stable environments for your software stack

If you have experience with conda, Spack is like conda, but far more robust and designed specifically for HPC clusters.

So, how do you use Spack?

2.1 - Spack Environments

We won't get into the nuances of spack here. Here's what you need to know: If you are not experienced building, installing, and maintaining software on your own, we **highly** recommend asking us to build a spack environment for you. Once you have a spack environment, you can run one command to load all of the libraries you need. We maintain some default environments for specific domains that contain the most commonly asked for packages. For example, our **default-ml** environment has torch, tensorflow, and many other machine learning packages that researchers typically use on the cluster. Another example is our **default-genomics** environment tailored towards genomics research.

The current list of default spack environments (as of 2025-10-16) is:

- default-astrophysics-x86_64-25031301
- default-genomics-x86_64-25091001
- default-lammps-x86_64-24091101
- default-ml-x86_64-25052701
- default-nlp-x86_64-25030601
- default-quantum-x86_64-24090301

You can load these environments using the command `$ spack env activate <environment_name>`.

When you're getting started, give these environments a try to see if they have all of the software you need. If you can't find the software you need in one of these environments, we will need to build a spack environment for you. To build a spack environment for you, we just need a list of the libraries you need and any specific versions that matter for your research.

Here's an example list of python libraries we received recently (the @ symbols mean *at a specific version number*):

```
# Python version
- python@3.9

# Python libraries where the version matters
- numpy@1.23.5

# Python libraries where the version doesn't matter
- pandas
- matplotlib
- scikit-learn
- amplpy
- statsmodels
```

Note: To request a custom spack environment, please include a list of the software you need, and any version numbers that are important, **in this form** [↗](#).

From this list, we built a spack environment called `vmd-23041101`. You can *activate* (load) this environment by running: `$ spack env activate <environment_name>`. In this example:

```
$ spack env activate vmd-23041101
```

Now, if we run `$ spack find` we can see all of the python libraries we have loaded:

```

$ spack find
==> In environment vmd-23041101
==> Root specs
py-amplpy      py-numpy@1.23.5  py-scikit-learn  python@3.9
py-matplotlib  py-pandas          py-statsmodels

==> Installed packages
-- linux-rhel7-skylake_avx512 / gcc@11.2.0 -----
autoconf@2.69                                mkfontdir@1.0.7                                py-pybind11@2.10.1
automake@1.16.5                              mkfontscale@1.1.2                             py-pyparsing@3.0.9
bdf2topcf@1.0.5                              nasm@2.15.05                                   py-pyproject-metadata@
0.6.1
berkeley-db@18.1.40                          ncurses@6.4                                    py-python-dateutil@2.
8.2
bzip2@1.0.8                                  ninja@1.11.1                                   py-pythran@0.12.0
ca-certificates-mozilla@2023-01-10          openblas@0.3.21                               py-pytz@2022.2.1
cmake@3.25.2                                 openssl@1.1.1t                                 py-requests@2.28.0
diffutils@3.8                                perl@5.36.0                                    py-scikit-learn@1.2.1
expat@2.5.0                                  pigz@2.7                                        py-scipy@1.10.0
font-util@1.3.2                              pkgconf@1.8.0                                 py-setuptools@59.4.0
fontconfig@2.13.1                            py-amplpy@0.8.6                               py-setuptools-scm@7.0.
5
fontproto@2.1.3                              py-ampltools@0.4.6                             py-six@1.16.0
freetype@2.11.1                              py-beniget@0.4.1                             py-statsmodels@0.13.2
gdbm@1.23                                    py-bottleneck@1.3.5                          py-threadpoolctl@3.1.0
gettext@0.21.1                              py-build@0.7.0                               py-tomli@2.0.1
gperf@3.1                                    py-certifi@2022.12.7                         py-typing-extensions@
4.3.0
inputproto@2.3.2                             py-charset-normalizer@2.0.12                  py-urllib3@1.26.12
kbproto@1.0.7                               py-contourpy@1.0.5                           py-versioneer@0.22
libbsd@0.11.7                               py-cppy@1.1.0                                 py-wheel@0.37.1
libffi@3.4.4                                py-cycler@0.11.0                             py-zipp@3.8.1
libfontenc@1.1.3                            py-cython@0.29.32                             python@3.9.15
libiconv@1.17                               py-flit-core@3.7.1                           qhull@2020.2
libjpeg-turbo@2.1.4                         py-fonttools@4.37.3                          re2c@2.2
libmd@1.0.4                                 py-future@0.18.2                             readline@8.2
libpng@1.6.37                              py-gast@0.5.3                                renderproto@0.11.1
libpthread-stubs@0.4                       py-idna@3.4                                   scrnsaverproto@1.2.2
libsigsegv@2.13                             py-importlib-resources@5.9.0                 sqlite@3.40.1
libx11@1.7.0                               py-joblib@1.2.0                              tar@1.34
libxau@1.0.8                               py-kiwisolver@1.3.2                          tcl@8.6.12
libxcb@1.14                                py-matplotlib@3.7.0                         tk@8.6.11
libxcrypt@4.4.33                           py-meson-python@0.11.0                      util-linux-uuid@2.38.1
libxdmcp@1.1.2                             py-numexpr@2.8.3                             util-macros@1.19.3
libxext@1.3.3                              py-numpy@1.23.5                             xcb-proto@1.14.1
libxfont@1.5.2                             py-packaging@23.0                            xextproto@7.3.0
libxft@2.3.2                               py-pandas@1.5.3                             xproto@7.0.31
libxml2@2.10.3                             py-patsy@0.5.2                              xtrans@1.3.5
libxrender@0.9.10                          py-pep517@0.12.0                            xz@5.4.1
libxscrnsaver@1.2.2                       py-pillow@9.2.0                             zlib@1.2.13
m4@1.4.19                                  py-pip@23.0                                 zstd@1.5.2

```

```
meson@1.0.0
==> 119 installed packages
py-ply@3.11
```

In the above output, you will see all of the libraries that we asked for (note that all python libraries start with `py-`), and all of their dependencies.

Note: You need to activate your spack environment inside your sbatch scripts and every time you log into the cluster.

2.2 - Spack via Interactive

To access spack environments via `sinteractive`:

1. Launch an `sinteractive` session following the documentation in [Part 1 of our Slurm Tutorial \(slurm_quick_start_tutorial.html\)](#).
2. Activate the spack environment you need: `spack env activate <environment_name>`.

2.3 - Spack via Batch

To access spack environments via `sbatch`:

1. Write an **sbatch** script following the documentation in [Part 1 of our Slurm Tutorial \(slurm_quick_start_tutorial.html\)](#).
2. In your **sbatch** script, between your configuration options (*i.e.* `#SBATCH` lines) and your code (*e.g.* `python my_script.py`), activate the spack environment you need: `spack env activate <environment_name>`.

2.4 - Further Reading on Spack

- [Spack Documentation](#) 
- [Basic Usage](#) 
- [Command Reference](#) 

3 - Apptainer

[Apptainer](#)  enables you to run containers on the cluster. We have installed Apptainer (using Spack) on the cluster, so you don't need to worry about that, you just need load Apptainer using Spack: `spack load apptainer`.

We will ensure that Apptainer is installed and working properly. That is where our support for Apptainer ends. Anything you do inside of a container is your responsibility.

Research Computing supports Apptainer, we do not support containers. Due to the non-standard and highly-customized nature of containers, we are not resourced to support them. You are

welcome to use Apptainer to run containers, but **you are responsible for supporting your containers**. We will not help you design, build, install, troubleshoot, or maintain containers. Spack is our supported package manager on the cluster. If you would like us to build and support a spack environment for you, please fill out [this form](#). Please be aware that spack environments typically take at least 2 weeks to build, install, and test.

3.1 - Containers

What is a container? Containers are just collections of software packages and their dependencies that are designed to be portable. You can [read more about that here](#).

This tutorial assumes that (1) you have a container that already has all of the software you need installed, and (2) you will just be running code on top of a container, not modifying the container. Modifying existing containers, building containers from scratch, and running as `root` inside of containers is **beyond the scope of this tutorial**. You are welcome to do those things, but **we are not resourced to help you do them**.

3.1.1 - Basic Example: Ubuntu Container

1. Load Apptainer with Spack: `spack load apptainer`.
2. Create an **apptainer** directory, which we will refer to as `<apptainer-dir>` in this documentation.
 - If you are sharing a container with your research project: `mkdir /shared/rc/<shared-dir>/apptainer`.
 - If you are not sharing a container: `mkdir /home/<username>/apptainer`.
3. Build a container using a container definition:
 - If you are pulling a container definition from **docker**: `apptainer build --fakeroot <apptainer-dir>/<container_name.sif> docker://docker.io/<container_name>`
 - For example, a basic **Ubuntu** container: `apptainer build --fakeroot <apptainer-dir>/ubuntu.sif docker://docker.io/ubuntu:jammy`
 - You can also write your own [definition file](#) for a custom container.
4. Run your container from your `<apptainer-dir>`: `./ubuntu.sif`

3.2 - Inside a Container Shell

When you launch a container shell, you will see a command prompt like this:

```
Apptainer>
```

If you run `pwd` from here, you will see the following (assuming your container is in a shared directory):

```
/shared/rc/<shared-dir>/apptainer
```

So, you still have access to your shared directory (or home directory) from inside your container. You can access and run files like you normally would.

If you want to access files inside the container, those files are located in `/etc` (within the container).

When you launch your container and run `whoami`, you will see the following:

```
<username>
```

This means that anything you do is with your normal permissions. You can read more about permissions in our [Linux & Bash Tutorial \(bash_tutorial.html\)](#).

3.3 - Apptainer via Interactive

To access containers via `sinteractive`:

1. Launch an `sinteractive` session following the documentation in [Part 1 of our Slurm Tutorial \(slurm_quick_start_tutorial.html\)](#).
2. Load Apptainer using Spack: `spack load apptainer`
3. If you did not request GPUs for your interactive session, run the following from your `<apptainer-dir>` to access a shell inside your container: `./ubuntu.sif`
 - This will change your command prompt to `Apptainer>` .
4. If you requested **GPUs** for your interactive session (e.g. `sinteractive --gres=gpu:a100:1`), you will need the `--nv` flag when you launch your container: `apptainer shell --nv ubuntu.sif`

Note: If the nvidia binaries are not installed inside of the container, the container will not be able to leverage GPUs.

3.4 - Apptainer via Batch

To access containers via `sbatch`:

1. Write an **sbatch** script following the documentation in [Part 1 of our Slurm Tutorial \(slurm_quick_start_tutorial.html\)](#).
2. In your **sbatch** script, between your configuration options (i.e. `#SBATCH` lines) and your code (e.g. `python my_script.py`), load Apptainer with Spack: `spack load apptainer` .

3. In your **sbatch** script, after you `spack load apptainer`, you can execute commands inside your container: `./ubuntu.sif <command>`

- If you requested **GPUs** for your job, you will need to run commands this way: `apptainer exec --nv ubuntu.sif <command>`
- **Note:** Commands that you run inside of the container are relative to root's home directory (`/root`).

Note: If the nvidia binaries are not installed inside of the container, the container will not be able to leverage GPUs.

Modifying existing containers, building containers from scratch, and running as root inside of containers is **beyond the scope of this tutorial**. You are welcome to do those things, but **we are not resourced to help you do them**.

Many use-cases for containers involve spinning up self-contained web applications that are always running. The Research Computing cluster is not designed for persistent (always running) workflows, nor is it open to the world for web applications.

3.4 - Further Reading on Apptainer

- [Apptainer Quick Start](#) 
- [What is a Container?](#) 
- [Building a Container](#) 
- [Running as `root` Inside Containers](#) 

4 - Home Directory Installs

You may opt to install software in your home directory on the cluster. You do not need `sudo` access to do so. There are many ways to install software in your home directory; the best way to do so will depend on the software you need. Some examples include [conda](#)  and [venv](#)  for Python, or [make](#)  and [cmake](#)  for C/C++.

As a reminder, if you opt to install software in your home directory, **you are supporting yourself**.

Research Computing does not support home directory installs. Due to the non-standard and highly-customized nature of home directory installs, we are not resourced to support them. You are welcome to install software in your home directory, but **you are responsible for supporting any software that you install yourself**. We will not help you build, install, debug, or maintain software in your home directory. Spack is our supported package manager on the cluster because it is a standard for HPC systems and prevents many of the challenges typically encountered with conda, pip, etc. If

you would like us to build and support a spack environment for you, please fill out [this form](#). Please be aware that spack environments typically take at least 2 weeks to build, install, and test.

For this tutorial, we will focus on conda because most researchers on the cluster are using python.

4.1 - Conda

4.1.1 - Installing Conda

1. Download the installer from [here](#): `wget <url_to_installer> .`
 - You want the “Miniconda3 Linux 64-bit” link for the version of Python you want. Don’t use the “Miniconda3 Linux-aarch64 64-bit”, “Miniconda3 Linux-ppc64le 64-bit”, or “Miniconda3 Linux-s390x 64-bit” links.
 - On 2022-08-24, this example worked: `wget https://repo.anaconda.com/miniconda/Miniconda3-py39_4.12.0-Linux-x86_64.sh`
2. Run the installer: `bash <installer_file> -b -p ~/<conda_directory_name> .`
 - Wait for the installer to finish.
3. Log out and back into `sporcs submit`.
4. Run `source ~/<conda_directory_name>/etc/profile.d/conda.sh` to add conda to your PATH.
 - **Note:** You will need to do this each time you login and in your sbatch scripts.
5. Make sure Conda is working correctly by running `conda list`.
 - You can also run `which conda` to see where your PATH thinks conda is.
6. You can check the version of conda installed by running `conda --version`.
 - You can see additional information by running `conda info`.

4.1.2 - Uninstalling Conda

If your conda gets messed up, you may need to completely uninstall it.

1. Remove the directory that you installed conda in: `rm -rf ~/<conda_directory_name> .`
2. Remove hidden conda files: `rm -rf ~/.condarc ~/.conda ~/.continuum .`
3. Remove any code that looks like this in your `~/.bashrc` :

- ```
>>> conda initialize >>>
!! Contents within this block are managed by 'conda init' !!
__conda_setup="$('/home/<username>/<conda_directory_name>/bin/conda' 'shell.b
ash' 'hook' 2> /dev/null)"
if [$? -eq 0]; then
 eval "$__conda_setup"
else
 if [-f "/home/<username>/<conda_directory_name>/etc/profile.d/conda.sh"]; t
hen
 . "/home/<username>/<conda_directory_name>/etc/profile.d/conda.sh"
 else
 export PATH="/home/<username>/sandbox/<conda_directory_name>/bin:$PATH"
 fi
fi
unset __conda_setup
<<< conda initialize <<<
```

#### 4. Logout and back into the cluster.

**Note:** We will not prevent you from modifying your `.bashrc` file, but we do not recommend it. Modified `.bashrc` files can lead to problems, such as software not loading/loading incorrectly, or being unable to login. If you choose to modify your `.bashrc` file, we can reset it to the default for you, **but we will not debug errors resulting from a modified `.bashrc`.**

### 4.1.3 - Creating a Conda Environment

1. Create a new environment: `conda create --name <environment_name> python=<version_number> .`
2. Verify that your environment exists: `conda info --envs .`
3. Activate your new environment: `conda activate <environment_name> .`
  - At this point, you will only have access to the python libraries installed in your environment.
4. Check that the correct version of Python was installed: `python --version .`
5. Install the python packages you need: `conda install <package_name> .`
  - You can search for packages using `conda search <package_name> .`
  - You can install a specific version like this: `conda install <package_name>=<version> .`
6. You can see the packages installed in your environment using `conda list .`
7. To stop using your environment, run `conda deactivate .`

**Note:** If you are installing libraries that can leverage **GPUs** (e.g. pytorch, tensorflow), cuda needs to be installed and loaded before installing/loading those libraries. You can try to install cuda on your own (not recommended), or you can load a version of cuda that we have installed with spack. Run `spack`

```
find -l cuda to find versions of cuda that we have installed. Then run spack load cuda@<version> /<build_hash> to load the version of cuda you need, e.g. spack load cuda@12.0.0 /4a5j4ca.
```

### 4.1.4 - Migrating Your Existing Conda Environment to the Cluster

#### Migrating from Linux

1. On your **local** computer, export a list of your packages: `conda list --explicit package_list.txt`.
2. Transfer `package_list.txt` to `sporcsuim` using one of the methods outlined in Section 3 of our [Storage Tutorial \(storage\\_tutorial.html\)](#).
3. On `sporcsuim`, recreate your environment: `conda create --name <environment_name> --file package_list.txt`.

#### Migrating from Windows/Mac

1. On your **local** computer, export your environment configuration: `conda env export environment.yml`.
2. Transfer `environment.yml` to `sporcsuim` using one of the methods outlined in Section 3 of our [Storage Tutorial \(storage\\_tutorial.html\)](#).
3. On `sporcsuim`, recreate your environment: `conda env create -f environment.yml`.

### 4.1.5 - Managing Your Conda Environment

To update the version of conda installed, run `conda update conda`.

To update a package installed in your conda environment, run `conda update <package_name>`.

To delete your environment, simply run: `conda env remove --name <environment_name>`.

## 4.2 - Conda via Interactive

To access conda environments via `sinteractive`:

1. Launch an `sinteractive` session following the documentation in [Part 1 of our Slurm Tutorial \(slurm\\_quick\\_start\\_tutorial.html\)](#).
2. Add conda to your `$PATH`: `source ~/<conda_directory_name>/etc/profile.d/conda.sh`
  - **Note:** If your conda environment requires cuda to be loaded via Spack, you will also need to load cuda here.
3. Activate your conda environment: `conda activate <environment_name>`

## 4.3 - Conda via Batch

To access conda environments via `sbatch`:

1. Write an **sbatch** script following the documentation in [Part 1 of our Slurm Tutorial \(slurm\\_quick\\_start\\_tutorial.html\)](#).
2. In your **sbatch** script, between your configuration options (*i.e.* `#SBATCH` lines) and your code (e.g. `python my_script.py`):
  - Add conda to your `$PATH`: `source ~/<conda_directory_name>/etc/profile.d/conda.sh`
  - Activate your conda environment: `conda activate <environment_name>`
  - **Note:** If your conda environment requires cuda to be loaded via Spack, you will also need to load cuda here.

## 4.4 - Further Reading on Conda

- [Conda Cheatsheet](#) 
- [Conda Documentation](#) 
- [Conda for Data Science Tutorial](#) 

# 5 - Licensed Software

If you would like to use licensed software on the cluster (and you have a valid license), please [submit a ticket](#)  and provide a link to the software, the version you would like, and the details of your license agreement. **Note:** Licensed software will only be installed and maintained with Spack. We will not support licensed software via Apptainer or Home Directory Installs.

## 6 - Summary

In this tutorial, you learned about three different ways to access the research software you need on the cluster: Spack, Apptainer, and Home Directory Installs. Now we will discuss pros and cons.

### 6.1 - Should I Use Spack, Apptainer, or Home Directory Installs?

Ultimately, that decision is up to you, but here are some helpful suggestions:

*Consider using **Spack** if...*

- Your software stack is mostly static, *i.e.* you use the same set of software for 6 or more months at a time.
- You would like to focus on your research instead of building, installing, and maintaining software.

- You are okay waiting **at least** two weeks for new software to be added to your environment.
- You are **not** comfortable building, installing, troubleshooting, and maintaining software on your own.

Consider using **Apptainer** if...

- You want a command line with *root* access in an interactive session. (**Note:** We will not show you how to do this.)
- You are trying to develop a portable software stack/workflow.
- You are in the prototyping phase of your research and don't know exactly what software libraries you need.
- You are comfortable building, installing, troubleshooting, and maintaining software on your own.

Consider using **Home Directory Installs** if...

- You are comfortable building, installing, troubleshooting, and maintaining software on your own.
- You need to use the most up-to-date version of a software library as soon as it is released.
- You are in the prototyping phase of your research and don't know exactly what software libraries you need.

**Note:** If you have questions about which method to use, feel free to come talk to us.

## 7 - Special Notes

### 7.1 - Mixing Spack and Conda

Spack and Conda both operate by managing your [unix environment](#) and changing your `$PATH`. For that reason, Spack and Conda don't get along. Using Spack for some software libraries and Conda for others often results in Conda getting confused and your conda environment no longer working. We highly discourage you from trying to mix Spack and Conda.

**Note:** If you are installing libraries that can leverage **GPUs** (e.g. pytorch, tensorflow), cuda needs to be installed and loaded before installing/loading those libraries. This is the one exception where mixing Spack and Conda is recommended. You can load a version of cuda that we have installed with spack. Run `spack find -l cuda` to find versions of cuda that we have installed. Then run `spack load cuda@<version> /<build_hash>` to load the version of cuda you need, e.g. `spack load cuda@12.0.0 /4a5j4ca`.

### 7.2 - Spanning Multiple Nodes

Spanning multiple nodes is supported via **Spack only**. You may be able to get MPI working with Apptainer or Home Directory Installs, but we are not resourced to help you do that.

**Help:** If there are any further questions, or there is an issue with the documentation, please **submit a ticket** [↗](#) or contact us on Slack for additional assistance.

**Tags:** [instructions \(tag\\_instructions.html\)](#) [spack \(tag\\_spack.html\)](#)

[Internal Documentation](#) [↗](#)

©2026 Copyright Research Computing. All rights reserved.

Page last updated: April 3, 2024

Site last generated: Mar 19, 2026

# RIT